S&DS 265 / 565

Introductory Machine Learning

Autoencoders and Transformers

April 6

Yale

Version of 2026/04/06 at 11:53

Reminders

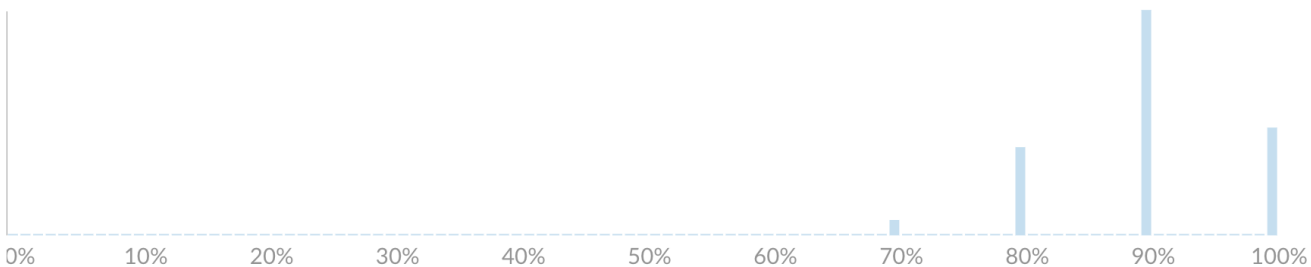
- Assignment 4 due Thursday
- Assignment 5 (last!) posted Thursday
- Final exam, May 4 at 2pm
- Review sessions TBA

Quiz 4

Quiz Summary

Section Filter ▾ | Student Analysis | Item Analysis

Average Score	High Score	Low Score	Standard Deviation	Average Time
90%	100%	70%	0.76	18:57



Quiz 4

How does Deep Q-Learning use deep neural networks?

Discrimination Index

37% answered correctly

A neural network with states as inputs is used to define a Q-function	76 respondents	59 %
A neural network is used to extract features of states	5 respondents	4 %
A neural network is used to define a value function		0 %
All of the above	48 respondents	**37 %** ✓
None of the above		0 %

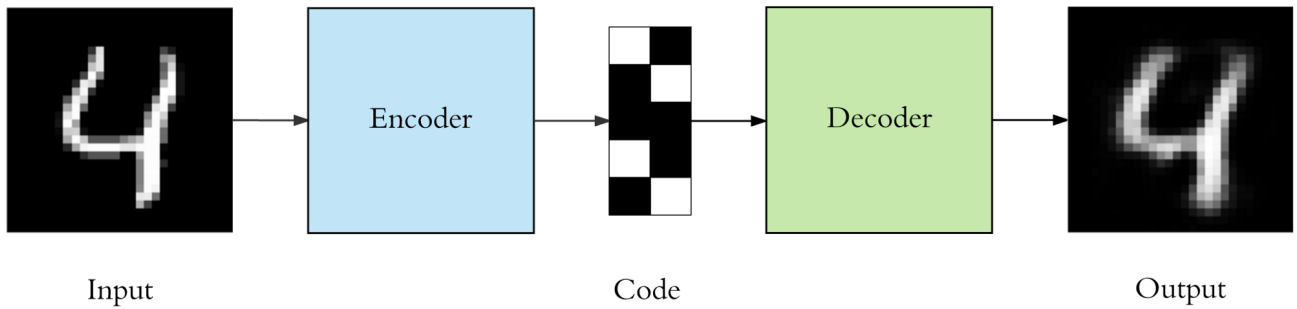
For today

- Autoencoders
- Transformers

Autoencoders

- Unsupervised learning methods
- Squeeze high dimensional data through a “bottleneck” of lower dimension
- Train to minimize reconstruction error

Autoencoders

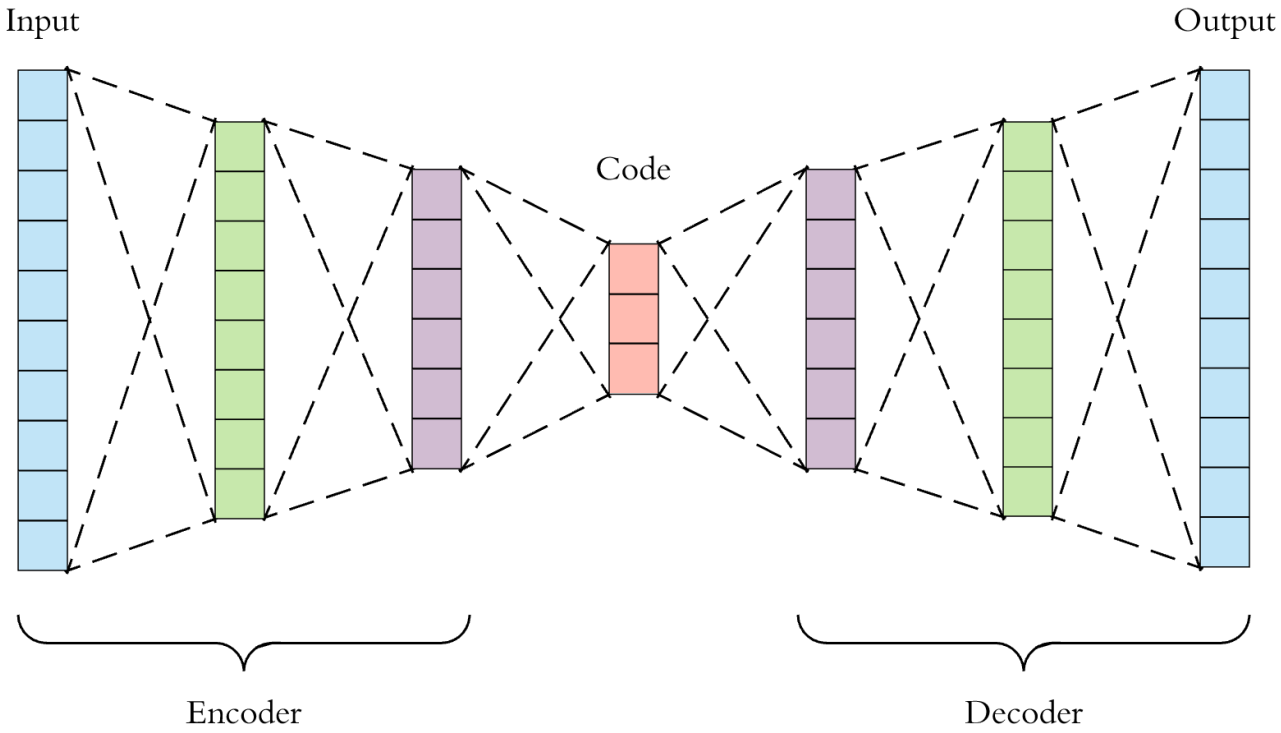


https://github.com/ardendertat/Applied-Deep-Learning-with-Keras

Important aspects

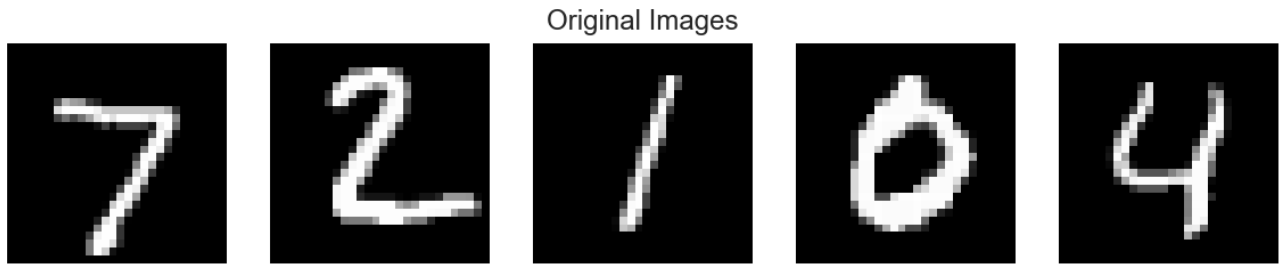
- Unsupervised: No labels used, discovers useful features of input
- Compression: Code reduces dimension of data
- Lossy: Input won’t be reconstructed exactly
- Trained: The compression algorithm is learned for specific data

Deep architecture



https://github.com/ardendertat/Applied-Deep-Learning-with-Keras

MNIST data



$$28 \times 28 = 784 = D$$

It cuts both ways...

`numpy`

`keras`

Implementation using Keras

```
input_size = 784
hidden_size = 128
code_size = 32

input_img = Input(shape=(input_size,))
hidden_1 = Dense(hidden_size, activation='relu')(input_img)
code = Dense(code_size, activation='relu')(hidden_1)
hidden_2 = Dense(hidden_size, activation='relu')(code)
output_img = Dense(input_size, activation='sigmoid')(hidden_2)

autoencoder = Model(input_img, output_img)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.fit(x_train, x_train, epochs=5)
```

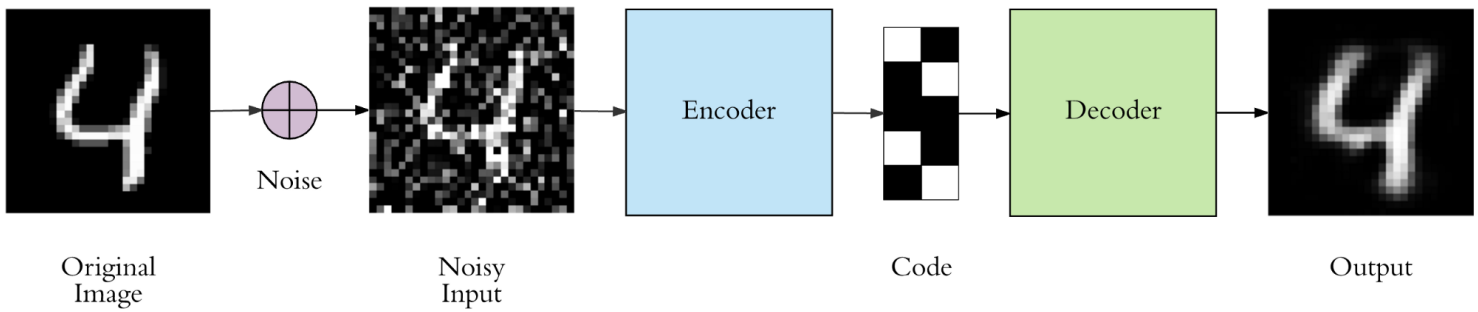
Adam optimizer

- Variant of stochastic gradient descent where separate learning rate (step size) is maintained for each network weight (parameter)
- Each step size adapted as learning progresses based on moments of the derivatives

---

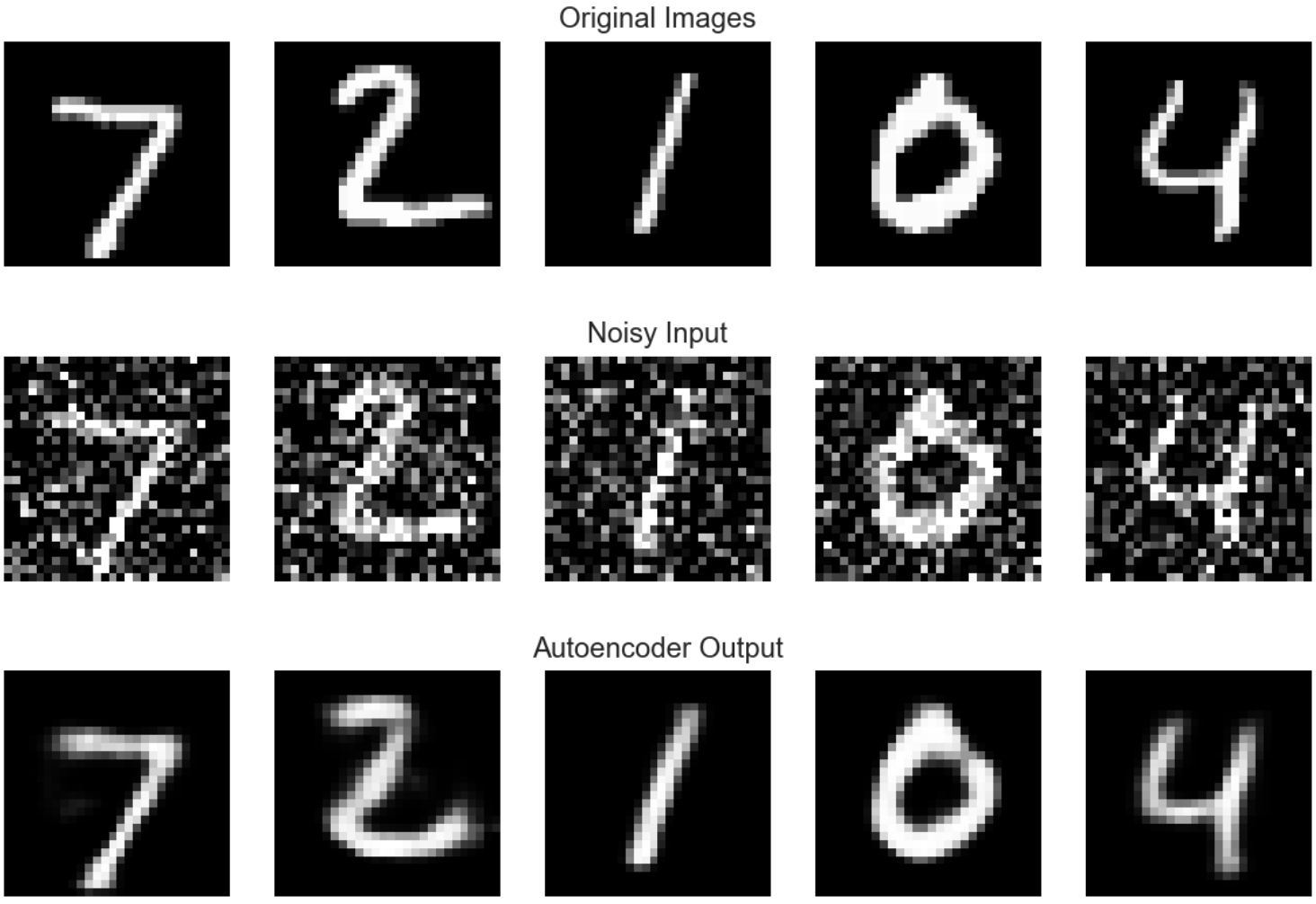
"Adam: A method for stochastic optimization," D. Kingma and J. Ba, https://arxiv.org/abs/1412.6980

Variant: Denoising autoencoder



- Feed in noisy data
- Train to match to denoised data

Example result on MNIST

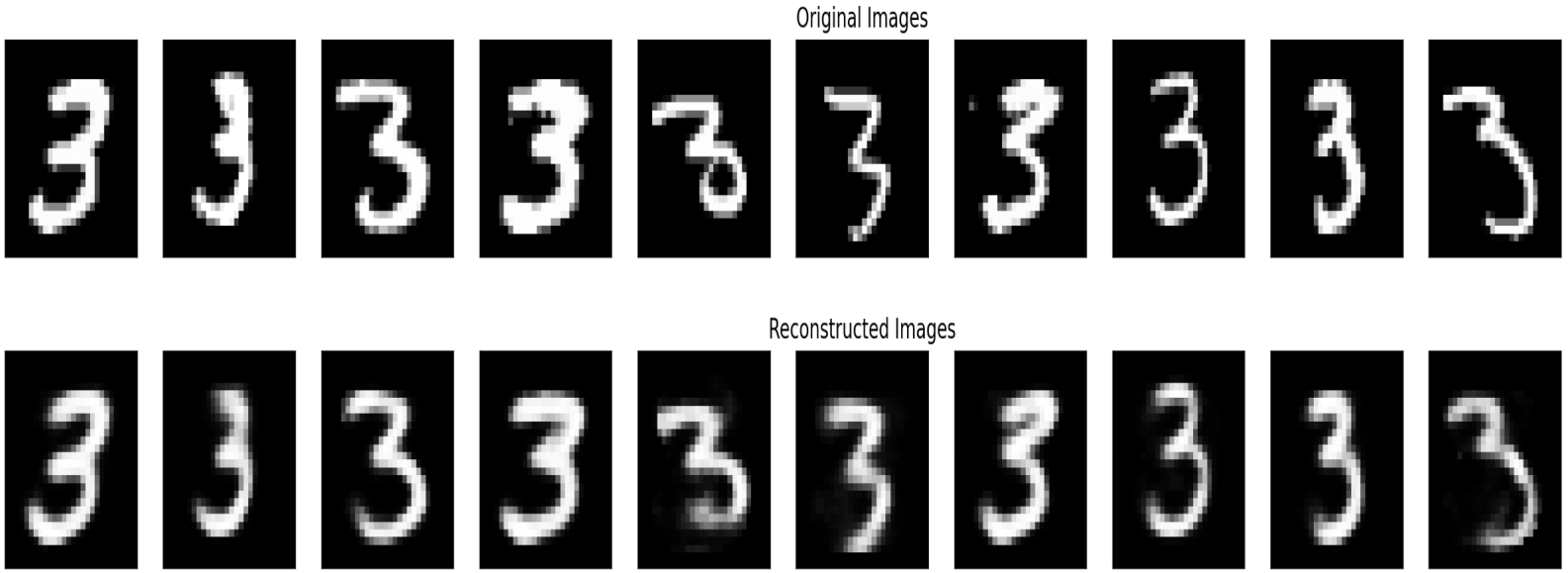


Example result on MNIST: 3's company

[30]
✓ 1s

```
plot_autoencoder_outputs(autoencoder, 10, (28, 28))
```

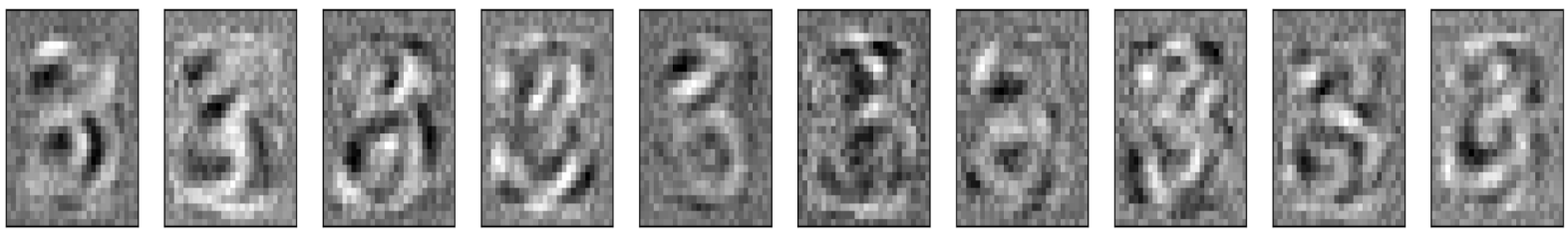
```
192/192 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```



Example result on MNIST: 3's company

[31]
✓ 0s

```
weights = autoencoder.get_weights()[0].T

n = 10
plt.figure(figsize=(n*2, 5))
for i in range(n):
    ax = plt.subplot(1, n, i + 1)
    plt.imshow(weights[i+0].reshape(28, 28))
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
```



Recall: The “Cat neuron”

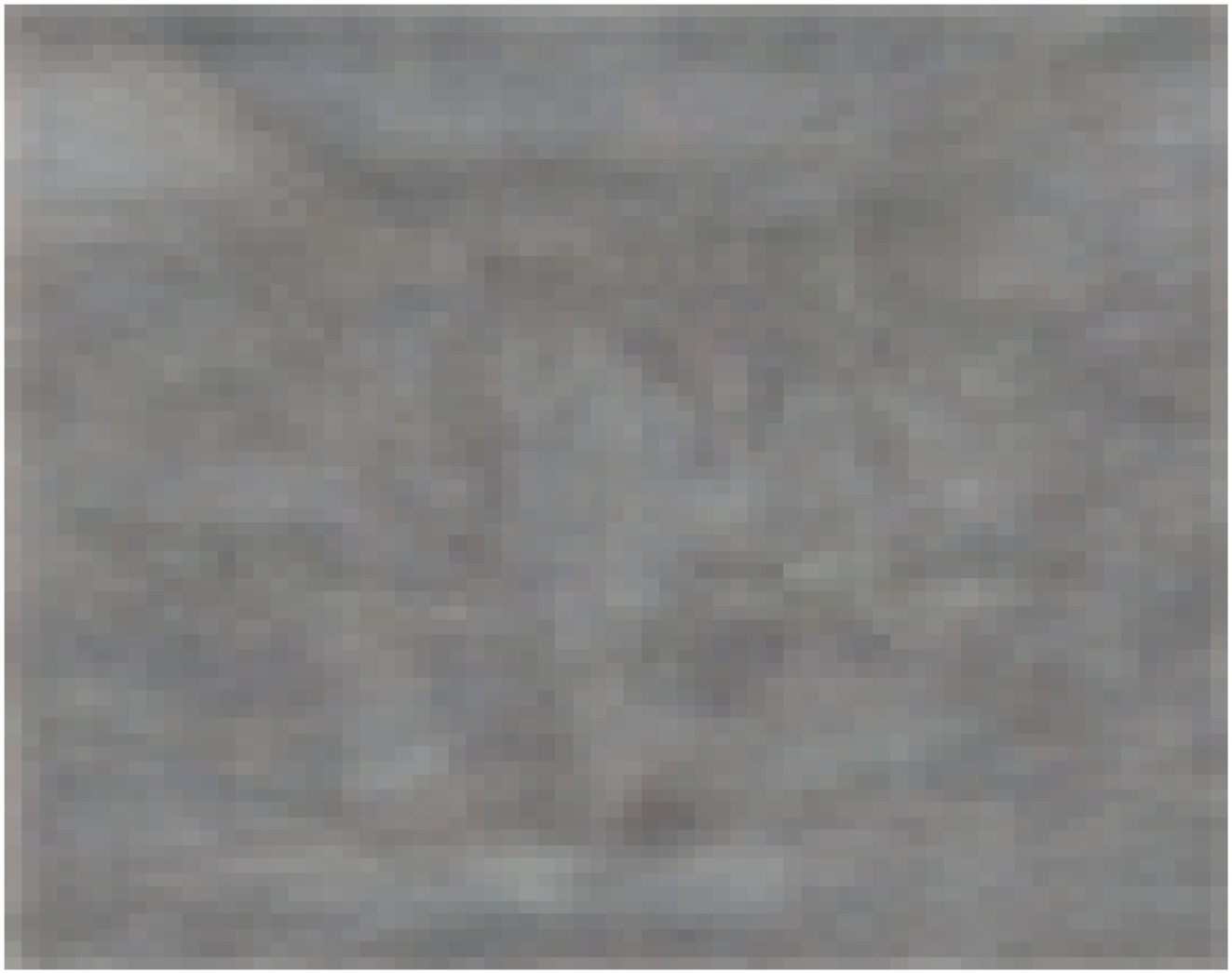
An Ahah! moment in AI history came in 2012

- Google Brain researchers (led by Andrew Ng and Jeff Dean) trained a “sparse autoencoder” neural net in an unsupervised fashion
- Data: 10 million *unlabeled* 200 $\times$ 200 pixel images from YouTube videos
- Trained for three days on 1,000 computers
- The network had a specific neuron that activated only for cat faces

---

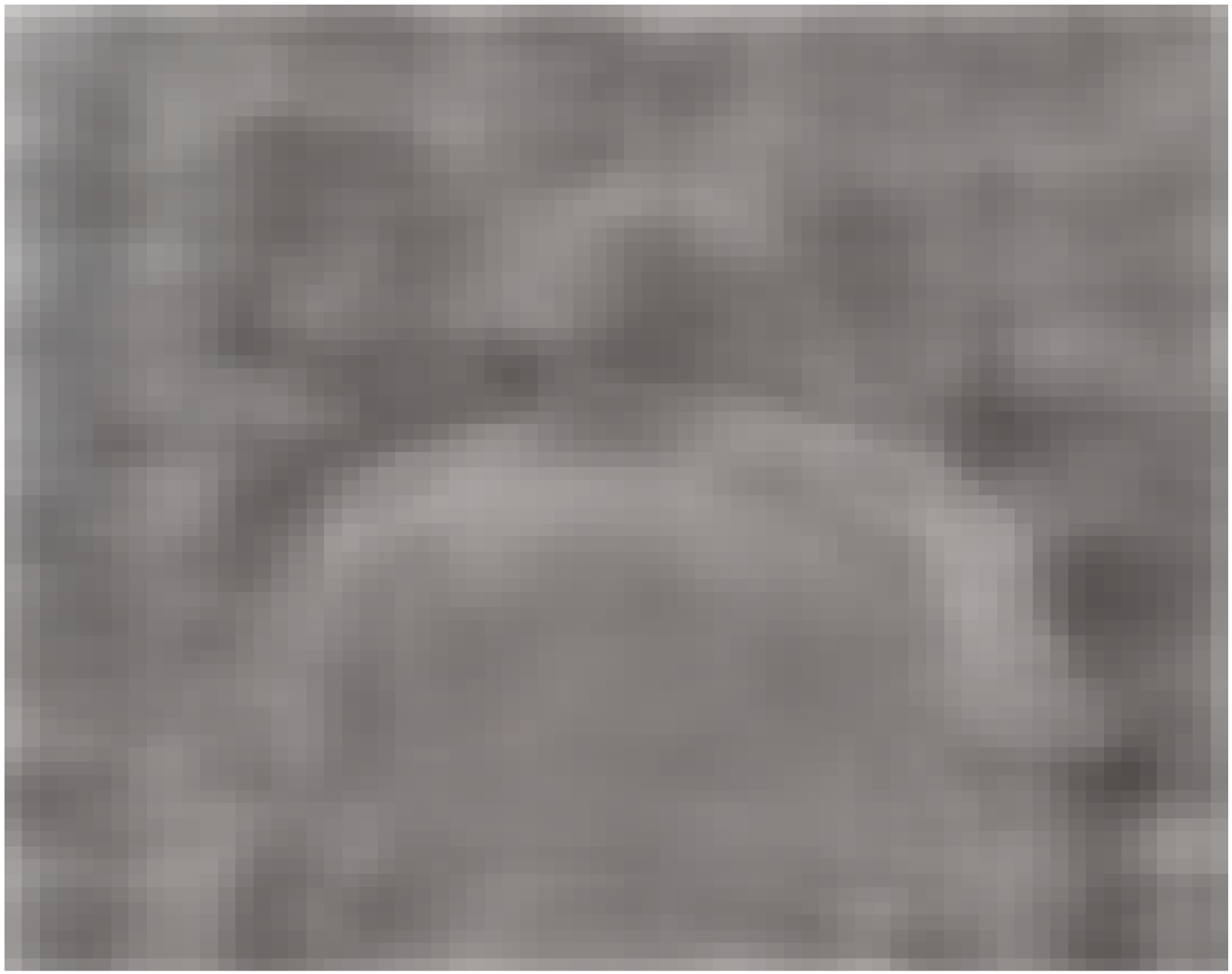
Le et al., “Building High-level Features Using Large Scale Unsupervised Learning,” 2012

Recall: The “Cat neuron”



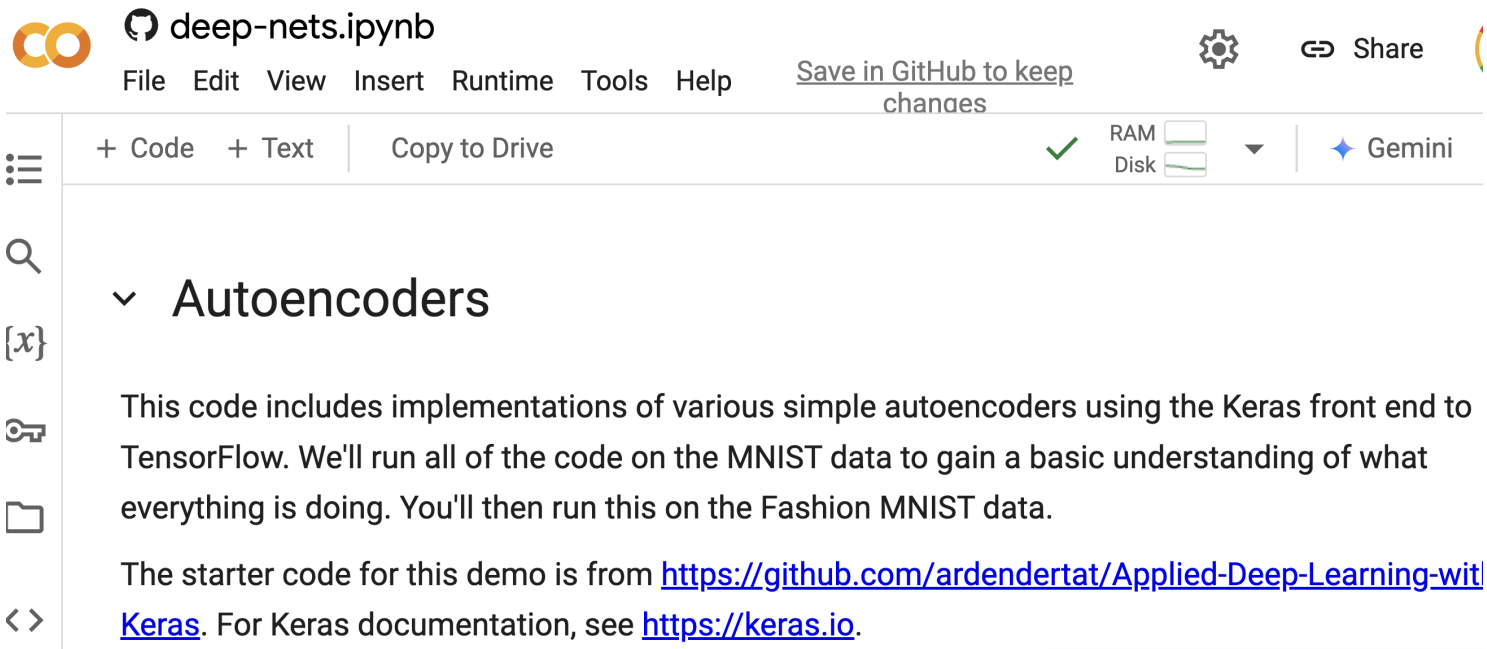
Le et al., “Building High-level Features Using Large Scale Unsupervised Learning,” 2012

Body neuron



Le et al., “Building High-level Features Using Large Scale Unsupervised Learning,” 2012

Demo


deep-nets.ipynb
File Edit View Insert Runtime Tools Help
Save in GitHub to keep changes
Share
+ Code + Text
Copy to Drive
RAM
Disk
Gemini
Autoencoders
This code includes implementations of various simple autoencoders using the Keras front end to TensorFlow. We'll run all of the code on the MNIST data to gain a basic understanding of what everything is doing. You'll then run this on the Fashion MNIST data.
The starter code for this demo is from https://github.com/ardendertat/Applied-Deep-Learning-wit
Keras. For Keras documentation, see https://keras.io.

Summary: Autoencoders

- Autoencoders compress the input and then reconstruct it
- Bottleneck forces extraction of useful features
- Will overfit and “memorize” the data
- Overfitting mitigated by denoising autoencoders

Large Language Models

https://support.apple.com/

We've all used language models many times today

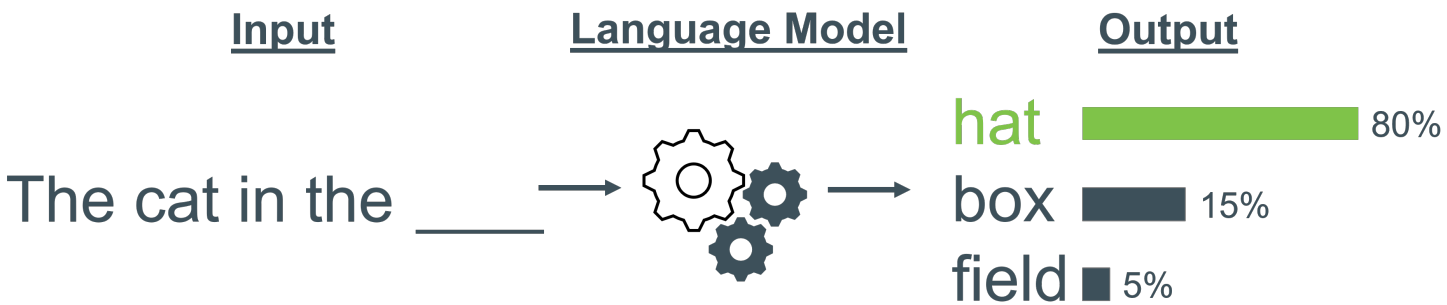
Home assistants

Many uses of language models

- Speech recognition (Siri, Alexa, etc.)
- Machine translation (Google translate)
- Texting
- Composing documents
- . . .

Text generation

- Words generated one-by-one
- A word is chosen by sampling from a probability distribution
- Result is purely synthetic text—generative AI



Remixing Noon

Text generated from Channel Skin by Jeff Noon

"The whole forest had been anesthetised, her temples wired, her senses stimulated, her eyes were not on Eva, not on Eva, not on Eva, not on anybody in that same realm, the land of dreams and nightmares."

Viability: 0.000000326%

https://chatbotslife.com/notes-on-remixing-noon-generative-text-and-markov-chains-84ff4ec23937

Language models

- Because it predicts the next word, a language model gives a way of *generating* text (through sampling)

$$
\begin{aligned}
&P(\text{“the whole forest had been anesthetized”}) = \\
&P(\text{“the”}) \times P(\text{“whole”} \mid \text{“the”}) \\
&\times P(\text{“forest”} \mid \text{“the whole”}) \\
&\times P(\text{“had”} \mid \text{“the whole forest”}) \\
&\times P(\text{“been”} \mid \text{“the whole forest had”}) \\
&\times P(\text{“anesthetized”} \mid \text{“the whole forest had been”})
\end{aligned}
$$

Modern language models

The algorithm assigns a “score” to possible next words w:

$$\text{score}(w;\ \overbrace{w_1, \ldots, w_n}^{\text{previous words}}\ )$$

w: possible next word

- score is computed as the similarity between two high-dimensional vectors—we’ll explain how!
- it is then converted into a probability

Key steps

- *Tokenize* the text into a fixed vocabulary
- *Embed* the tokens into high-dimensional vectors
- *Mix* the embeddings in the context using “Attention”
- *Map* the resulting vectors using a neural network

Tokenization

The vocabulary problem

- Word embeddings we saw constructed for 400,000 word vocabulary
- We each know $\approx$40K words
- What if a word is not in the vocabulary?
- Need to be able to process–and generate–arbitrary text

Tokenization and the BPE method were first developed in the 2019 OpenAI GPT-2 paper, Radford et al. “Language models are unsupervised multitask learners”

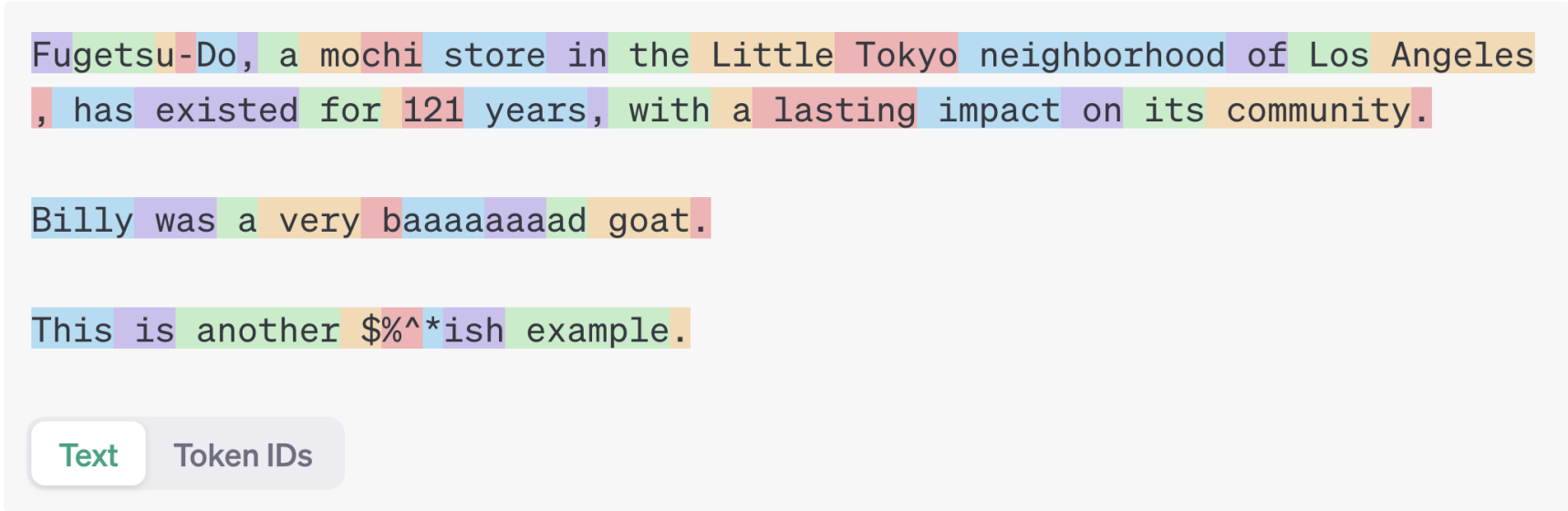
Tokenization

```
Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los
Angeles, has existed for 121 years, with a lasting impact on its
community.

Billy was a very baaaaaaaaad goat.

This is another $%^*ish example.
```

This is what we see

platform.openai.com/tokenizer

Tokenization



This is how the tokenizer groups bytes into tokens

platform.openai.com/tokenizer

Tokenization

```
[77845, 19209, 84, 12, 5519, 11, 264, 4647, 14946, 3637, 304, 279, 15013,
27286, 12818, 315, 9853, 12167, 11, 706, 25281, 369, 220, 7994, 1667, 11,
449, 264, 29869, 5536, 389, 1202, 4029, 382, 97003, 574, 264, 1633, 293,
29558, 33746, 329, 54392, 382, 2028, 374, 2500, 400, 46999, 9, 819, 3187,
382]
```

Text | Token IDs

This is what the LLM sees—the “atoms” of an LLM

platform.openai.com/tokenizer

Byte Pair Encoding

Algorithm 1 Byte Pair Encoding (BPE)

procedure BPE(Sequence S, Steps K)
    Initialize: Each byte in S is a token in $\{0, \ldots, 255\}$
    $V \leftarrow 256$
    for $i = 1$ **to** K **do**
        Find pair (s, t) with the highest adjacent co-occurrence count in S
        Replace all occurrences of (s, t) in S with the new token V
        $V \leftarrow V + 1$
    end for
    return S
end procedure

Tokenizer properties

- *Reversible:* Can convert from tokens back to original text
- *General:* Works on arbitrary text, even very different from training
- *Compressive:* Each token is about 4 bytes
- *Statistical:* Will break strings into frequently occurring pieces

Byte level handles any language

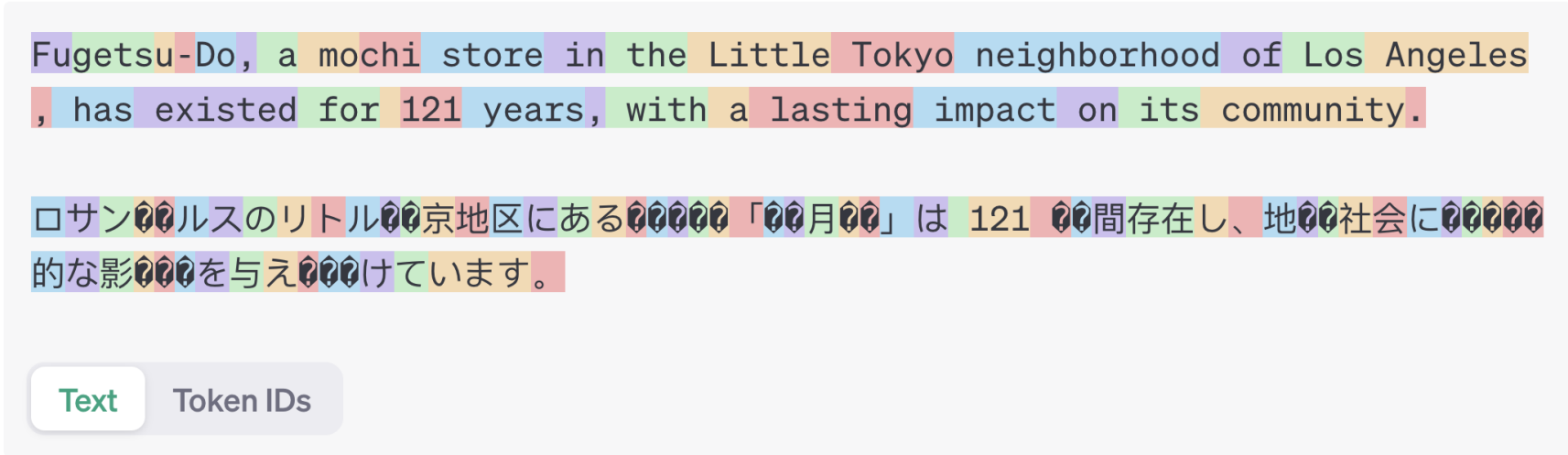
Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

ロサンゼルスのリトル東京地区にある餅屋「風月堂」は 121 年間存在し、地域社会に永続的な影響を与え続けています。

This is what we see

platform.openai.com/tokenizer

Byte level handles any language



This is how the tokenizer groups bytes into tokens

platform.openai.com/tokenizer

Byte level handles any language

```
[77845, 19209, 84, 12, 5519, 11, 264, 4647, 14946, 3637, 304, 279, 15013,
27286, 12818, 315, 9853, 12167, 11, 706, 25281, 369, 220, 7994, 1667, 11,
449, 264, 29869, 5536, 389, 1202, 4029, 382, 42634, 60868, 16073, 3484,
120, 33710, 22398, 16144, 37823, 20251, 33710, 14276, 109, 47653, 30590,
24775, 20230, 30591, 30369, 165, 97, 227, 23602, 233, 13177, 19817, 101,
9953, 45144, 224, 10646, 15682, 220, 7994, 75677, 112, 56965, 48706,
15024, 5486, 30590, 35722, 253, 61337, 38093, 20230, 30320, 116, 163,
90183, 9554, 26854, 58322, 165, 253, 123, 30512, 58318, 58942, 163,
90183, 76622, 38144, 61689, 3490]
```

Text | Token IDs

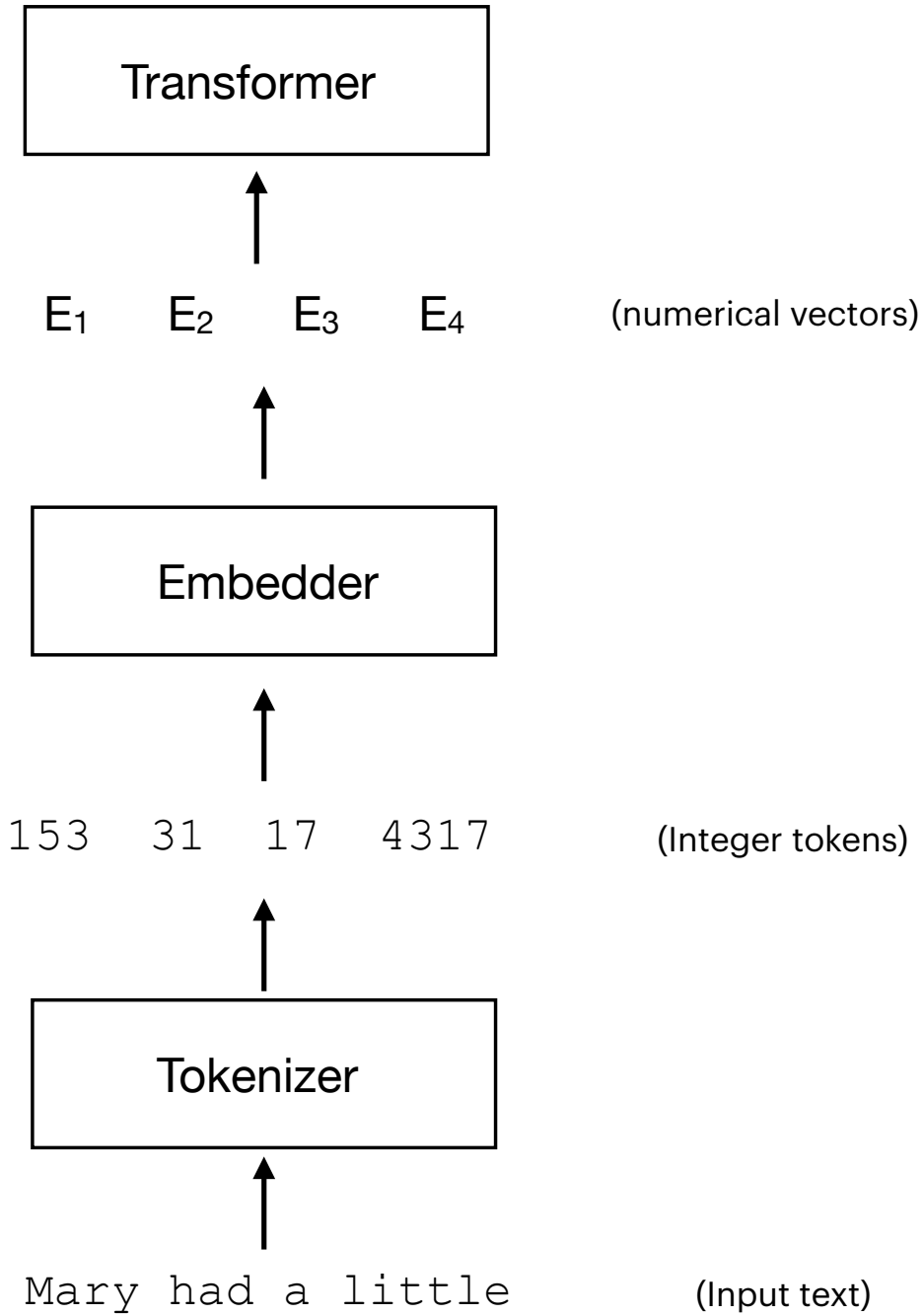
This is what the LLM sees—the “atoms” of an LLM

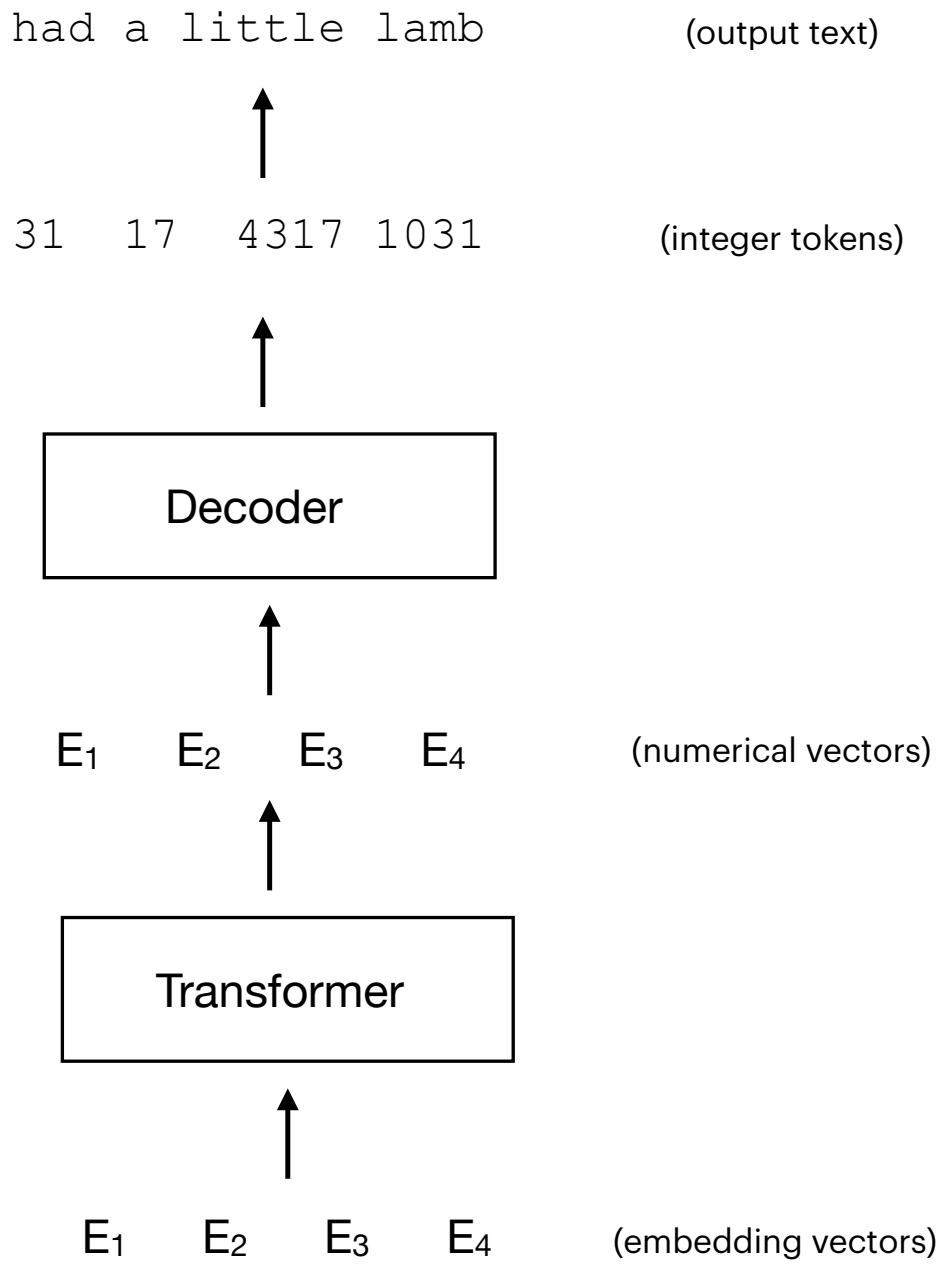
platform.openai.com/tokenizer

Tokenizer issues

Tokenization can lead to troublesome artifacts in LLMs:

- Very general since computed from raw byte streams (Unicode)
- Uneven treatment of different languages
- Failure to recognize equivalence of different uses of same word
- Poor handling of numbers


Transformer
E1 E2 E3 E4
(numerical vectors)
Embedder
153 31 17 4317
(Integer tokens)
Tokenizer
Mary had a little
(Input text)


had a little lamb
(output text)
31 17 4317 1031
(integer tokens)
Decoder
E1 E2 E3 E4
(numerical vectors)
Transformer
E1 E2 E3 E4
(embedding vectors)

Word embeddings (review)

Puzzle

- Words are “symbols” — `banana`, `apple`, `Microsoft`
- How do we represent words in a form that a neural network can process?

Nietzsche: “Words are but symbols for the relations of things to one another and to us; nowhere do they touch upon absolute truth.”

Intuition

- Represent words by vectors to capture similarity in usage
- Similar words will appear with similar words (self-referential)
- Words are characterized by the words they co-occur with
- Purely statistical

Constructing embeddings

`word2vec`:

- Based on a very simple language model

Mikolov et al. “Distributed representations of words,” (2014); “Efficient representations of words in vector space” (2015)

Constructing embeddings

`word2vec`:

- Based on a very simple language model
- Predict *surrounding* words from current word, rather than the next word

Mikolov et al. “Distributed representations of words,” (2014); “Efficient representations of words in vector space” (2015)

Constructing embeddings

`word2vec`:

- Based on a very simple language model
- Predict *surrounding* words from current word, rather than the next word
- A model of *nearby* words $p_{\text{near}}(w_2 \mid w_1)$

Mikolov et al. “Distributed representations of words,” (2014); “Efficient representations of words in vector space” (2015)

Constructing embeddings

Language model is formed by

$$p_{\text{near}}(w_2 \mid w_1) \propto \exp\left(\varphi(w_2)^T \varphi(w_1)\right)$$

where the vectors $\varphi(w)$ are the parameters of the model

- Embeddings are "tuned" to predict nearby words

Mikolov et al. at Google (2014) "Distributed representations of words."

Here $a^T b$ means the "inner product" of the vectors: $a^T b = a_1 \cdot b_1 + a_2 \cdot b_2 + \cdots + a_d \cdot b_d$. Two vectors are similar if their inner product is large. It's related to the cosine of the angle between them in the high dimensional space.

```
embedding['copyright']
```

```
array([ 0.28292 , -1.0518  ,  0.17169 , -0.19509 ,  1.1078  ,  0.35123 ,
       -1.1153  , -0.7574  ,  0.43468 , -0.23271 ,  0.56626 , -0.32403 ,
        0.20586 , -0.7068  , -0.37967 ,  0.2811  ,  0.66518 ,  0.066218,
       -0.49656 , -0.44746 , -0.76732 , -0.69151 ,  1.0989  ,  0.68884 ,
        1.0099  , -0.099243,  0.86778 , -0.74706 , -0.53393 , -0.98153 ,
        0.33981 ,  0.87678 ,  0.55539 ,  0.25211 , -0.093136, -0.12268 ,
       -0.18995 , -0.5596  ,  0.022301, -0.65089 ,  0.15131 ,  0.69039 ,
       -0.25731 , -0.10418 ,  0.3187  , -0.92036 , -0.080776, -0.72957 ,
       -0.062274, -0.30242 ,  1.108   ,  0.45579 , -0.24216 , -0.17086 ,
        0.47928 , -0.38471 ,  0.55145 , -0.56831 ,  2.1103  , -0.40532 ,
       -0.35658 ,  0.30065 , -1.1101  , -0.40778 ,  0.65017 , -0.10852 ,
        0.97684 ,  0.30761 ,  0.043331, -0.54005 , -0.36475 ,  0.72474 ,
        0.18027 ,  0.41623 , -0.32136 , -0.942   , -0.87021 ,  0.009036,
       -1.6118  ,  0.085464,  0.46778 , -0.99888 ,  0.97935 ,  0.54841 ,
       -0.51551 ,  0.32104 , -0.57828 , -0.43939 ,  0.20083 ,  0.48005 ,
       -0.094059, -0.27191 ,  0.04519 , -0.8264  ,  0.86619 ,  0.76831 ,
       -0.006007, -0.12633 ,  0.25886 , -0.37395 ], dtype=float32)
```

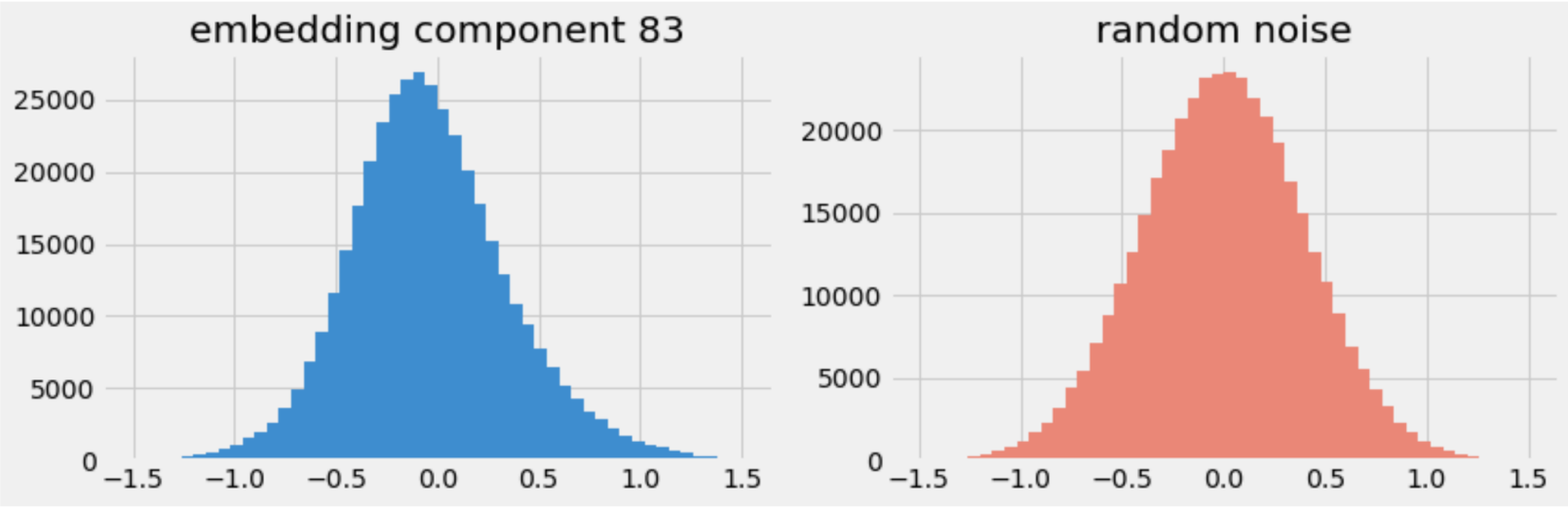
You can experiment with embeddings here:
https://colab.research.google.com/github/YData123/sds265-fa24/blob/main/demos/embeddings/embeddings.ipynb

```
embedding.most_similar('copyright', topn=10)
```

```
[('infringement', 0.7312791347503662),
 ('copyrights', 0.6773107051849365),
 ('patent', 0.6438419818878174),
 ('patents', 0.6030457019805908),
 ('licensing', 0.602853536605835),
 ('piracy', 0.602036416530609 1),
 ('copyrighted', 0.597329020500 1831),
 ('rights', 0.589721620082 8552),
 ('privacy', 0.587808489799 4995),
 ('ipr', 0.585008263587 9517)]
```

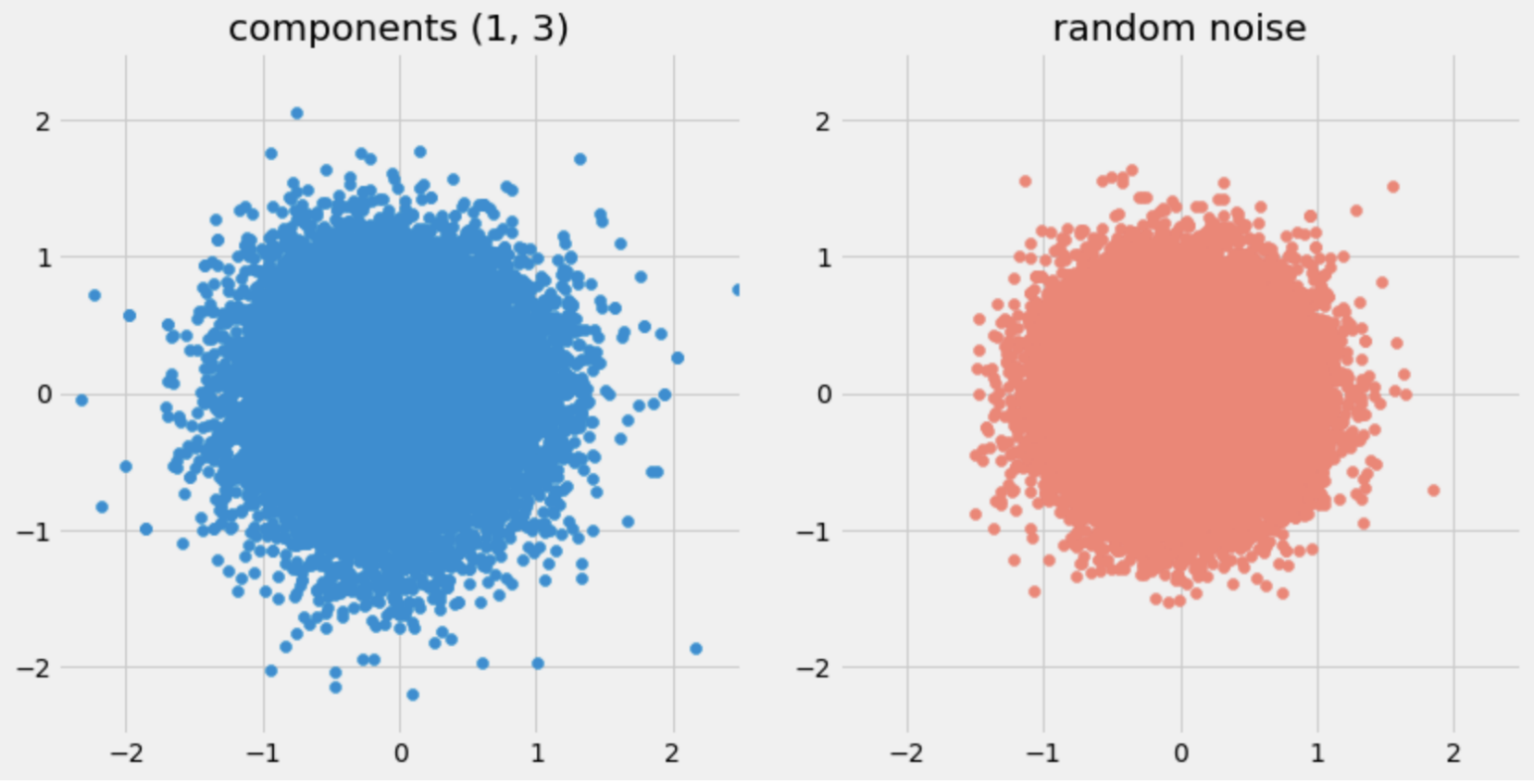
You can experiment with embeddings here:
https://colab.research.google.com/github/YData123/sds265-fa24/blob/main/demos/embeddings/embeddings.ipynb

Embeddings look like random noise



But they encode a huge amount of useful information!

Embeddings look like random noise



But they encode a huge amount of useful information!

Analogies

Leads to vector representations of words with interesting properties.

For example, analogies:

`king` is to `man` as ? is to `woman`

Analogies

Leads to vector representations of words with interesting properties.

For example, analogies:

`king` is to `man` as ? is to `woman`

`Paris` is to `France` as ? is to `Germany`

Analogies

Leads to vector representations of words with interesting properties.

For example, analogies:

$\texttt{king}$ is to $\texttt{man}$ as ? is to $\texttt{woman}$

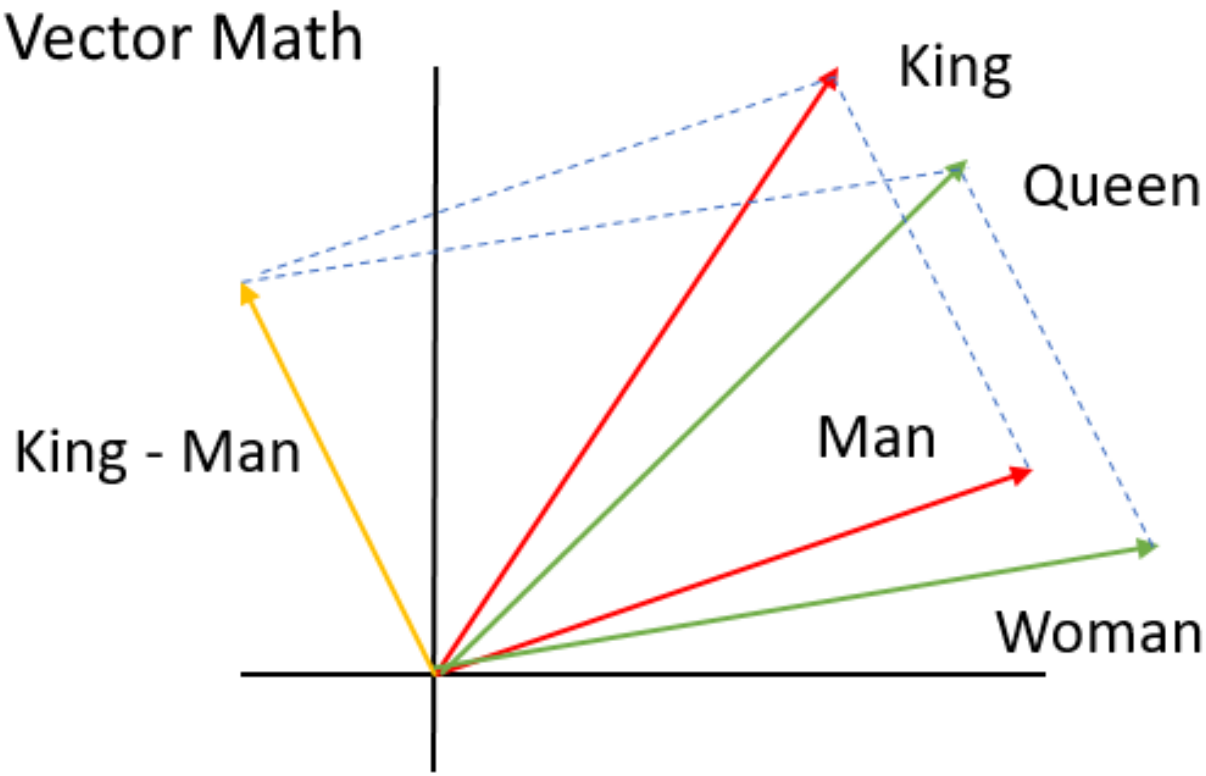
$\texttt{Paris}$ is to $\texttt{France}$ as ? is to $\texttt{Germany}$

$$\varphi(\texttt{king}) - \varphi(\texttt{man}) \stackrel{?}{\approx} \varphi(\texttt{queen}) - \varphi(\texttt{woman})$$

$$\widehat{w} = \arg\min_{w} \|\varphi(\texttt{king}) - \varphi(\texttt{man}) + \varphi(\texttt{woman}) - \varphi(w)\|^2$$

Does $\widehat{w} = \texttt{queen}$?

Word geometry



Word geometry

$\varphi(\texttt{king}) - \varphi(\texttt{man}) + \varphi(\texttt{woman})$:

```
queen
elizabeth
isabella
empress
throne
princess
```

Learned Analogies

Table 8: *Examples of the word pair relationships, using the best word vectors from Table 4 (Skip-gram model trained on 783M words with 300 dimensionality).*

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Mikolov et al., “Distributed representations of words,” (2014); “Efficient representations of words in vector space” (2015)

Embeddings encode societal bias

$\varphi(\texttt{worker}) - \varphi(\texttt{man}) + \varphi(\texttt{woman})$:

nurse
migrant
immigrant
child
nurses
pregnant
teacher

$\varphi(\texttt{worker}) - \varphi(\texttt{woman}) + \varphi(\texttt{man})$:

working
factory
farmer
mechanic
laborer

Trained on 6 billion words of Wikipedia

Embeddings encode societal bias

φ(scientist) - φ(man) + φ(woman):	anthropologist sociologist psychologist geneticist biochemist
φ(scientist) - φ(woman) + φ(man):	geologist engineer astronomer mathematician science

Trained on 6 billion words of Wikipedia

Embeddings encode societal bias

φ(smart) - φ(girl) + φ(boy):

wise
better
guy
kind
good
kid

φ(smart) - φ(boy) + φ(girl):

sexy
pretty
incredibly
cute
exciting
funny

Trained on 6 billion words of Wikipedia

Summary: Word embeddings

- Word embeddings are vector representations of words, learned from cooccurrence statistics
- The models can be built using simple language modeling
- Surprising semantic relations are encoded in linear relations
- Embeddings are the “ground floor” representations in LLMs